

Work Breakdown Structure

Project Name: ARCC (AI Resume and Career Coach)

Course: ENSF 400

Section: 01

Group #: 13

Group Members + UCIDs:

Ryan Graham - UCID: 30171130

Fateh Ali - UCID: 30123431

Rabab Rahman - UCID: 30179444

Muhammad Zain - UCID: 30162484

Benny Ndabaramiye - UCID: 30166945

Team Role:

Ryan Graham: Project Management, Database Control and Cross-Section Support

Fateh Ali: Backend Developer

Rabab Rahman: Frontend Developer

Muhammad Zain: AI/API Integration Developer

Benny Ndabaramiye: Frontend Developer/Backend Support

1. Work Breakdown Structure

1.0 ARCC System Development

1.1 Project Management

1.1.1 Project Planning Framework

1.1.2 Requirements Traceability Framework

1.1.3 Risk and Dependency Management Framework

1.2 System Design

1.2.1 Requirements Baseline and Use Case Model

1.2.2 Architecture and Component Design

1.2.3 UI/UX Design Framework

1.3 Frontend System

1.3.1 User Interface Framework

1.3.2 Resume Upload Interface

1.3.3 Job Description Input Interface

1.3.4 Analysis Results Interface

1.3.5 Interview Simulator Interface

1.3.6 User Dashboard Interface

1.4 Backend System

1.4.1 REST API Framework

1.4.2 Resume Parsing Module

1.4.3 Resume-to-Job Matching Engine

1.4.4 Skill Gap Analysis Engine

1.4.5 Resume Improvement Suggestion Engine

1.4.6 Interview Simulation Engine

1.4.7 Speech-to-Text Integration Module

1.4.8 User Authentication Module

1.5 Database System

1.5.1 Database Schema Design

1.5.2 User Data Storage Module

1.5.3 Resume and Analysis Storage Module

1.6 External Service Integration

1.6.1 LLM API Integration Module

1.6.2 Security and Encryption Framework

1.7 Quality Attributes

1.7.1 Security Compliance Framework

1.7.2 Performance Optimization Module

1.7.3 Reliability and Recovery Mechanism

1.7.4 Scalability Architecture Framework

1.7.5 Accessibility Compliance Framework

2. WBS Dictionary

1.1 - Project Management:

1.1.1 Project planning, tracking, and coordination

- Scope: Plan iterations, interrelate on teamwork, track progress, be in charge of handling certain risks tied to third-party dependencies (LLMs/STTs)
- Deliverables: Project plan, sprint plan or board, risk lists and milestone check-ins
- Acceptance criteria: Clear backlog for all SRS features and cross-cutting requirements; risks will be documented for external service dependency.
- Dependencies/Assumptions: External LLM API availability, stable hosting, and cross-browser compatibility
- Section Lead: **Ryan Graham**

1.1.2 Requirements Traceability Framework

- Scope: Establish and maintain traceability between SRS requirements and project artifacts (WBS items, user stories/tasks, design components, and test cases); document and manage requirement changes and their impacts.
- Deliverables: Requirements Traceability Matrix (RTM) mapping SRS → WBS IDs → tasks/user stories → test cases; requirements change log with impact notes.
- Acceptance criteria: Every SRS requirement maps to at least one WBS work package and a verification approach; any changes to requirements are recorded with rationale and impacted components identified.
- Dependencies/Assumptions: SRS is available and remains reasonably stable; the team follows consistent naming/ID conventions across requirements, WBS items, and tests.
- Section Lead: [Ryan Graham](#)

1.1.3 Risk and Dependency Management Framework

- Scope: Identify, document, monitor, and mitigate project risks and dependencies (technical, schedule, and external services such as LLM/STT and hosting); define contingency plans and escalation triggers for high-impact risks.
- Deliverables: Risk and dependency register (likelihood/impact, mitigations, owners); mitigation/contingency plan items; milestone risk reviews and dependency readiness check-ins (e.g., external API availability).
- Acceptance criteria: Major risks and third-party dependencies (LLM/STT, hosting) are documented with mitigation steps and owners; the risk register is reviewed and updated at each milestone/check-in; contingency actions exist for critical external-service risks.
- Dependencies/Assumptions: External LLM/STT services are accessible for development/testing; a hosting environment is available; integration timelines are known early enough to manage schedule impacts.
- Section Lead: [Ryan Graham](#)

1.2 - System Design

1.2.1 Requirements baseline and use cases

- Scope: Translating SRS into user stories plus acceptance criteria; define as “done” for each feature and non-functional requirements
- Deliverables: Requirements backlog, and an acceptable criteria list
- Acceptance Criteria: Each feature from “Resume Upload and Parsing” to “Interview Practice Simulator”
- Section Lead: [Benny Ndabaramiye](#)

1.2.2 Architecture and component design

- Scope: Define system architecture (frontend, backend, database, AI/LLM, STT) and flow of data
- Deliverables: Diagram of the architecture, component responsibilities, sequence and data flow series
- Acceptance Criteria: The architecture reflects the described standalone web app with integrations of AI/LLM and speech recognition
- Section Lead: [Rabab Rahman](#)

1.2.3 UI/UX design (navigation and screens)

- Scope: Define UX flow and UI layouts for the required pages
- Deliverables: Wireframes or prototypes, UI navigation map(s)
- Acceptance Criteria: All listed screens and navigation elements represented
- Section Lead: [Rabab Rahman](#)

1.3 Frontend System

1.3.1 User Interface Framework

- Scope: Define and implement the overall frontend structure and shared UI patterns (routing/navigation, layout, reusable components, form validation patterns, and state management) to support all frontend screens consistently.
- Deliverables: Frontend routing/navigation structure; shared layout (header/footer/sidebar); reusable UI component library (buttons, inputs, cards); common form validation utilities; base styling/theme guidelines.
- Acceptance criteria: Frontend screens use consistent layout and reusable components; navigation between pages works reliably; shared components render correctly across supported browsers and screen sizes.
- Dependencies/Assumptions: React tooling/environment is set up; UI/UX design guidelines or wireframes are available; the team agrees on component conventions and routing structure.
- Section Lead: [Rabab Rahman](#)

1.3.2 Resume Upload Interface

- Scope: Build the UI for uploading a resume (PDF/DOCX), including file selection, validation, upload progress/status, error handling, and submission to the backend for parsing and storage.
- Deliverables: Resume upload page/component; client-side validation (file type/size); upload status/progress indicators; error/success messaging; integration with backend upload endpoint.

- Acceptance criteria: Users can upload a valid resume file and receive confirmation; invalid files (wrong type/too large/empty) show clear error messages; upload results are passed forward to the next step in the workflow.
- Dependencies/Assumptions: Backend resume upload/parsing endpoint exists; supported file formats and size limits are defined; stable network and browser file upload support.
- Section Lead: [Rabab Rahman](#)

1.3.3 Job Description Input Interface

- Scope: Create the UI for entering or pasting the job description, validating input, and submitting it to the backend for analysis along with the uploaded resume.
- Deliverables: Job description input page/component (textarea + controls); input validation (non-empty, reasonable length); error/success messaging; integration with backend analysis endpoint.
- Acceptance criteria: Users can submit a job description successfully; empty/invalid input is blocked with clear errors; submitted job description is stored/available for the analysis workflow.
- Dependencies/Assumptions: Backend endpoint for job description submission/analysis exists; expected input limits are defined; resume upload step provides required resume reference/session data.
- Section Lead: [Benny Ndabaramiye](#)

3.4 Analysis Results Interface

- Scope: Implement the results UI to display matching score, keyword/skills overlap, missing skills/gaps, and AI-generated suggestions in a readable and actionable format.
- Deliverables: Results page/component; sections for score/summary, matched vs missing skills, recommendations; basic filtering/expand-collapse for readability; integration to fetch results from backend.
- Acceptance criteria: Results load and display correctly after analysis; key outputs (score, gaps, suggestions) are clearly visible and understandable; UI handles partial/empty results gracefully with informative messaging.
- Dependencies/Assumptions: Backend returns a defined results payload (score, matched skills, missing skills, suggestions); UI/UX design expectations for results layout exist.
- Section Lead: [Benny Ndabaramiye](#)

1.3.5 Interview Simulator Interface

- Scope: Develop the UI for interview practice (question prompt, user response input via text and/or optional voice), session flow controls, and display of feedback/results from the backend.
- Deliverables: Interview simulator page/component; question display + “next” controls; response input (text and optional microphone controls); feedback display section; integration with backend interview/feedback endpoint(s).

- Acceptance criteria: Users can start an interview session, respond to prompts, and view feedback; UI clearly indicates recording/processing states if voice is enabled; errors (no mic permission, API failure) are handled with clear messages.
- Dependencies/Assumptions: Backend interview simulation/feedback functionality exists; any STT/voice features depend on browser permissions and external services; interview question set/format is defined.
- Section Lead: [Benny Ndabaramiye](#)

1.3.6 User Dashboard Interface

- Scope: Build the dashboard UI to allow users to view/manage saved resumes, past analyses, and interview sessions; provide navigation to start new analyses or practice interviews.
- Deliverables: Dashboard page/component; lists/cards for saved items (resume/analysis/interview history); basic actions (view, start new, delete/clear if supported); integration with backend user/history endpoints.
- Acceptance criteria: Dashboard loads user-specific data and displays history accurately; navigation to key workflows (upload, analysis, interview) works; dashboard handles “no history” state cleanly.
- Dependencies/Assumptions: User accounts/session management and history storage exist (or mocked if not implemented); backend endpoints to fetch user/history data are available.
- Section Lead: [Benny Ndabaramiye](#)

1.4 Backend System

1.4.1 REST API Framework

- Scope: Design and implement the backend service structure and REST API endpoints to support frontend workflows (upload resume, submit job description, run analysis, retrieve results, manage users and history); define request/response schemas and error handling.
- Deliverables: Flask backend project structure; API route definitions; request/response models (schemas); centralized error handling and logging; API documentation (endpoint list, payload examples).
- Acceptance criteria: API endpoints respond consistently with correct status codes; invalid inputs return clear error responses; endpoints integrate with database/services as required by the workflow.
- Dependencies/Assumptions: Backend framework (Flask) and environment are set up; API contracts with frontend are agreed; database connection is available.
- Section Lead: [Fateh Ali](#)

1.4.2 Resume Parsing Module

- Scope: Accept uploaded resume files (PDF/DOCX), validate and extract text content, normalize formatting, and produce a structured representation (e.g., sections/skills/keywords) for downstream analysis.
- Deliverables: Resume file validation (type/size); text extraction pipeline; normalization/cleaning utilities; parsed resume output structure (raw text + extracted fields/keywords); integration with upload endpoint.
- Acceptance criteria: Valid resumes are parsed into usable text/structured output; invalid files fail gracefully with informative errors; parsed output is consistent and usable by matching and skill-gap components.
- Dependencies/Assumptions: File extraction libraries/tools are available; supported formats and size limits are defined; a storage strategy for uploaded files/text is available.
- Section Lead: [Fateh Ali](#)

1.4.3 Resume-to-Job Matching Engine

- Scope: Compare parsed resume content against the job description to compute a matching score and identify overlaps (keywords/skills/requirements); generate a match summary for the results UI.
- Deliverables: Matching algorithm implementation, scoring logic, extraction of matched keywords/skills, and match summary payload returned through analysis endpoint.
- Acceptance criteria: Engine returns a reproducible match score and a list of matched elements; handles missing/short inputs without crashing; outputs align with the defined API response contract.
- Dependencies/Assumptions: Resume parsing output and job description input are available; the scoring approach is defined (rule-based, embedding similarity, or hybrid); consistent tokenization/normalization methods.
- Section Lead: [Fateh Ali](#)

1.4.4 Skill Gap Analysis Engine

- Scope: Identify missing or weak skills/requirements by comparing job description requirements against resume content; categorize gaps (technical vs non-technical) and provide actionable gap list.
- Deliverables: Gap identification logic; categorization rules; gap list payload (missing skills/requirements + context); integration into analysis response.
- Acceptance criteria: Returns a clear list of missing skills/requirements with categories; avoids false “missing” results for clearly present skills; handles ambiguous cases gracefully with “uncertain/needs review” when appropriate.
- Dependencies/Assumptions: Job description parsing/keyword extraction is available; resume parsing output is available; shared skill taxonomy/keywords list (if used) is agreed.
- Section Lead: [Fateh Ali](#)

1.4.5 Resume Improvement Suggestion Engine

- Scope: Generate tailored resume improvement recommendations based on match and skill gaps (e.g., wording suggestions, missing skills to highlight, structure improvements); optionally use an LLM API to draft suggestions.
- Deliverables: Suggestion generation logic (rule-based and/or LLM prompt pipeline); recommendations payload (bullet suggestions + prioritized actions); safety/guardrails to avoid fabricated experience suggestions.
- Acceptance criteria: Suggestions are relevant to the job description and grounded in the user's resume content; recommendations are actionable and formatted for display; system avoids adding unverifiable claims and flags items as "skills to develop" when not present.
- Dependencies/Assumptions: External LLM API access (if used); prompt templates and output format are defined; rate limits and error handling for external calls are implemented.
- Section Lead: [Muhammad Zain](#)

1.4.6 Interview Simulation Engine

- Scope: Provide interview practice workflow: generate interview questions based on job description/resume; accept user responses; produce feedback (strengths, improvement areas, STAR structure); and optionally provide scoring.
- Deliverables: Question generation logic (rule-based and/or LLM); feedback generation module; session flow support (start/next/end); API endpoints for interview session actions and results.
- Acceptance criteria: Users can receive questions and submit responses; feedback is returned in a structured, readable format; the system handles session state and failures cleanly (timeouts, API errors).
- Dependencies/Assumptions: Job description and resume context are available; external LLM access if used; defined feedback rubric and response format.
- Section Lead: [Fateh Ali](#)

1.4.7 Speech-to-Text Integration Module

- Scope: Enable optional voice input by converting recorded audio to text and passing transcriptions into the interview simulation flow; support microphone permission and audio handling requirements.
- Deliverables: STT service integration (API client); audio upload/processing endpoint (if needed); transcription output payload; error handling for permission/quality/timeouts.
- Acceptance criteria: Audio input produces accurate enough transcription for interview responses; failures return clear error messages and allow fallback to text input; latency remains acceptable for demo use.

- Dependencies/Assumptions: External STT service availability and credentials; supported audio formats are defined; frontend can capture and submit audio.
- Section Lead: **Muhammad Zain**

1.4.8 User Authentication Module

- Scope: Implement user account authentication and session handling (register/login/logout), password handling, and access control for protected endpoints (e.g., dashboard/history).
- Deliverables: Authentication endpoints; password hashing and validation; session/token handling; protected route middleware; basic user account models integrated with database.
- Acceptance criteria: Users can register and log in securely; protected endpoints require valid authentication; invalid credentials are handled safely without leaking sensitive info.
- Dependencies/Assumptions: Database schema for users exists; chosen auth approach (sessions/JWT) is agreed; security requirements (password policy) are defined.
- Section Lead: **Fateh Ali**

1.5 Database System

1.5.1 Database Schema Design

- Scope: Design the database schema to support ARCC features, including users, resumes, job descriptions, analysis results, interview sessions, and related metadata; define relationships, constraints, and indexing strategy.
- Deliverables: ERD/schema diagram; SQL schema definitions (tables, keys, constraints); indexing plan for common queries (user history, latest analysis); migration/setup script.
- Acceptance criteria: Schema supports required data flows (upload → analysis → history); primary/foreign keys and constraints enforce integrity; schema can be created successfully in the target DB environment.
- Dependencies/Assumptions: Database technology (e.g., MySQL) is available; data requirements from SRS/workflows are stable; team agrees on naming conventions and retention requirements.
- Section Lead: **Ryan Graham**

1.5.2 User Data Storage Module

- Scope: Implement database access logic to store and retrieve user account data and user-specific history needed for the dashboard (profiles, authentication-related fields, saved sessions).
- Deliverables: User table access layer/DAO; CRUD operations for user records; queries for user profile/history retrieval; integration with authentication module.
- Acceptance criteria: User records are stored and retrieved correctly; queries return the correct user-specific data; failures (duplicate accounts, missing users) are handled with safe error responses.

- Dependencies/Assumptions: Authentication module and user schema exist; database connection and credentials are configured; password data is stored securely (hashed).
- Section Lead: [Ryan Graham](#)

1.5.3 Resume and Analysis Storage Module

- Scope: Store and retrieve uploaded resume metadata/text, job descriptions, match scores, skill gaps, suggestions, and interview session outputs so users can view past results in the dashboard.
- Deliverables: Tables/fields for resumes, job descriptions, analysis results, and interview sessions; data access layer/DAO methods; queries for listing and viewing saved analyses; optional retention/cleanup rules.
- Acceptance criteria: Resume/job description/analysis results are persisted and linked to the correct user; users can retrieve past analyses reliably; stored data matches the API response format and supports the results/dashboard UI.
- Dependencies/Assumptions: Schema for resume and analysis entities exists; file storage strategy is defined (store text/metadata in DB and files separately if needed); backend endpoints consume/produce consistent IDs.
- Section Lead: [Ryan Graham](#)

1.6 External Service Integration

1.6.1 LLM API Integration Module

- Scope: Integrate an external LLM service to support resume improvement suggestions and interview simulation (prompt design, request/response handling, rate-limit handling, and fallback behavior when the service is unavailable).
- Deliverables: LLM API client wrapper/service layer; prompt templates for suggestions and interview feedback; standardized response parsing/formatting; retry/backoff + timeout handling; basic usage logging (success/failure).
- Acceptance criteria: Backend can successfully call the LLM and return structured outputs that match the API contract; failures (timeouts/rate limits/invalid responses) are handled gracefully with clear error messaging or fallback outputs; prompts avoid generating ungrounded claims not present in the resume/JD.
- Dependencies/Assumptions: Valid API credentials and service availability; agreed prompt/output format; rate limits and cost constraints are known; network access is stable in the deployment environment.
- Section Lead: [Muhammad Zain](#)

1.6.2 Security and Encryption Framework

- Scope: Implement security controls for handling user data and external service credentials, including encryption for sensitive data at rest/in transit, secure secret management, and input validation to reduce common web risks.

- Deliverables: HTTPS/TLS configuration (where applicable); secure storage of API keys/secrets (env vars/secret manager); encryption/hashing approach for sensitive fields; validation/sanitization utilities; security checklist for endpoints (auth-protected routes, least-privilege access).
- Acceptance criteria: Sensitive credentials are not hard-coded and are stored securely; user passwords are hashed (not stored in plaintext); data transmission is protected (TLS where applicable); inputs are validated to reduce injection/abuse risks; security measures do not break core workflows.
- Dependencies/Assumptions: Hosting environment supports TLS and secure secret storage; authentication module and database schema support secure storage practices; team follows secure coding conventions.
- Section Lead: **Ryan Graham**

1.7 Quality Attributes

1.7.1 Security Compliance Framework

- Scope: Ensure the system meets defined security requirements through secure coding practices, access control, validation, and basic compliance checks (auth-protected routes, least privilege, safe error handling, and secure handling of user data).
- Deliverables: Security checklist/compliance notes; secure endpoint review (auth/roles where applicable); input validation and error-handling standards; dependency vulnerability review (basic).
- Acceptance criteria: Protected endpoints require authentication; sensitive data is not exposed in logs or error messages; common injection/abuse vectors are mitigated through validation/sanitization; security checks are documented and reviewed.
- Dependencies/Assumptions: Authentication and encryption frameworks exist; team follows secure development practices; deployment supports secure configuration.
- Section Lead: **Ryan Graham**

1.7.2 Performance Optimization Module

- Scope: Optimize response times and resource usage for major workflows (upload/parsing, analysis generation, results retrieval) by improving query efficiency, caching where useful, and reducing unnecessary API calls.
- Deliverables: Performance baseline notes (simple measurements); optimized database queries/indexes; caching strategy (if applicable); improved API response efficiency (payload size, batching).
- Acceptance criteria: Key endpoints respond within acceptable time for demo use; repeated operations avoid redundant computation where feasible; database queries are efficient for user history and results retrieval.

- Dependencies/Assumptions: Database schema and endpoints are implemented; performance targets are defined (demo-ready thresholds); access to profiling/logging tools.
- Section Lead: **Muhammad Zain**

1.7.3 Reliability and Recovery Mechanism

- Scope: Improve system robustness by handling failures gracefully (external API downtime, parsing failures, DB connection issues) and enabling recovery through retries, fallbacks, and safe state management.
- Deliverables: Centralized error handling; retry/backoff policies for external services; fallback behaviours (e.g., text-only mode if STT fails); basic recovery procedures and logs.
- Acceptance criteria: The system does not crash on common failure cases; errors return actionable messages and allow users to continue or retry; external service failures are handled with timeouts and fallbacks; and reliability behaviours are documented.
- Dependencies/Assumptions: External services (LLM/STT) can fail intermittently; logging/monitoring is available; backend architecture supports centralized error handling.
- Section Lead: **Muhammad Zain**

1.7.4 Scalability Architecture Framework

- Scope: Define and apply scalability considerations so the system can handle increased users/requests (stateless API design where possible, database scalability considerations, and external service rate-limit management).
- Deliverables: Scalable architecture notes (statelessness, separation of concerns); rate-limit handling strategy for LLM/STT; database indexing and connection pooling plan; deployment considerations (horizontal scaling readiness).
- Acceptance criteria: Backend services are designed to be stateless or minimally stateful; database access patterns support growth (indexes, pooling); external API usage is controlled to avoid exceeding rate limits; the scalability approach is documented.
- Dependencies/Assumptions: The deployment environment can scale services; the database supports indexing and connection pooling; known external API limits.
- Section Lead: **Ryan Graham**

1.7.5 Accessibility Compliance Framework

- Scope: Ensure the frontend meets basic accessibility standards (keyboard navigation, readable contrast, semantic HTML, labels/ARIA where needed) so core workflows are usable by a broader set of users.
- Deliverables: Accessibility checklist; updates to UI components (labels, focus states, ARIA attributes); basic keyboard navigation and screen-reader-friendly structure; accessibility testing notes.

- Acceptance criteria: All primary forms and controls are keyboard accessible; input fields have labels; the UI provides clear focus indicators; major pages meet basic accessibility expectations (WCAG-informed); and issues are documented/fixed where feasible.
- Dependencies/Assumptions: Frontend UI framework and components exist; the team has access to basic testing tools (browser audits); UI design supports accessible patterns.
- Section Lead: [Rabab Rahman](#)

3. WBS diagram

